Wave 1 / M1.5 Sweep — Notebook Patch & Post-hoc Analysis Cell

DAMPS-MMHCL Phase 2 Section 8.2 — Execution Hygiene Instructions

Replace the final notebook cell and append a post-hoc analysis cell

DAMPS-MMHCL Research Team

June 17, 2026

Contents

1 Motivation

While inspecting the most recent M1.5 sweep logs we found two execution-hygiene defects that are far more serious than the early-stopping question they were originally raised under:

1. **The runs are not a real M1.5 LogQ sweep.** The attached notebook (`file:103`) contains *zero* occurrences of `enable_logq`, `logq_scale`, or `M1.5`. The launched command lines still consist solely of the nine `--damps_*` Phase 1 flags. Worse, the log timestamps read `2026-05-12`, i.e. they are recycled May outputs — not fresh runs with the LogQ correction. The Recall@20 $\approx$ 0.088–0.091 numbers therefore reflect the bare rev45 baseline; no LogQ delta has been applied yet.
2. **Seeds are corrupted by an implicit `float` cast.** The logs contain `seed=1.404632241e+09`, `seed=1.569402867e+09`, i.e. the seed was coerced to scientific-notation `float` before reaching `--seed`. This silently round-trips through 32-bit rounding and breaks bit-level reproducibility of the sweep. The fix is `str(int(seed))` at the call site.

The early-stopping logic itself is sound. If you want runs to terminate more cleanly after convergence (to save GPU time), you may lower `early_stopping_patience` from **30** to **20**; this is optional and not required for correctness.

2 Acceptance Criteria for the Patched M1.5 Sweep

- Every launched `train.py` command must contain all five LogQ flags, namely:

```
--enable_logq 1
--logq_mode {laplace, raw, sqrt}
--logq_beta 1.0
--logq_scale {0.05, 0.1, 0.3, 1.0}
--logq_clip 5.0
```

- Every `--seed` argument must be the string form of an integer (e.g. `1404632241`), never scientific notation.
- The M1.5 gate must be evaluated on `BEST_Test_Recall@20` (the test-set snapshot taken at the validation-recall peak epoch), *not* on any mid-run epoch printout.
- The per-run logger directory — whose name embeds `seed` and `ablation_target` — must encode `(mode, scale, seed)` so configurations do not overwrite each other's log files.
- Validation-recall curves for every (mode, scale, seed) must be plotted post-hoc so it can be visually verified that no run was terminated within < 15 epochs of its recall peak.

3 Part 1 – Replace the Final Notebook Cell (M1.5 Sweep Driver)

Open the attached notebook and overwrite the contents of the last code cell (the cell titled "Section 8.2 – Wave 1 / M1.5 sweep driver") with the script reproduced verbatim in Appendix A (`cell30_m15_fixed.py`). The patched cell embodies five concrete changes relative to the previous revision:

Verbatim hyper-parameters (from `Phase 2 Section 8.2`) used by the patched cell:

- `LOGQ_SCALES = [0.05, 0.1, 0.3, 1.0]` (four-point scale sweep); `LOGQ_MODES = ["laplace"]` (extend to `["raw","sqrt"]` for the full 24-run sweep).
- `N_SEEDS = 5`; `LOGQ_BETA = 1.0`; `LOGQ_CLIP = 5.0`.
- Acceptance gate: mean BEST_Test_R@20 ≥ 0.0925 **and** min BEST_Test_R@20 ≥ 0.0890.

Defect in previous cell	Fix applied in the patched cell
LogQ flags missing – launched command line is rev45 Phase 1 only	The constant `BACKBONE_FLAGS` now omits LogQ; LogQ activation (`--enable_logq 1`, `--logq_mode`, `--logq_beta`, `--logq_scale`, `--logq_clip`) is appended unconditionally inside `run_one(mode, scale, seed)`.
`seed` coerced to `float` (∼`1.4e+09`)	`_as_int_seed = lambda s: int(round(float(s)))` is applied when the seed list is bound, and `str(int(seed))` is used at the call site for `--seed`.
Sweep configs collide in the per-run log directory because `ablation_target` is the constant `wave1_m15_logq`	`ablation_target = f"m15_logq_{mode}_s{scale}_seed{seed}"`. This propagates into `train.py`'s `_experiment_paths()` and yields a unique directory per configuration.
M1.5 gate previously read mid-run epoch metrics	Regex parser now anchors on the `BEST_Test_*` family (`BEST_Test_Recall@20`, `BEST_Test_NDCG@20`, `BEST_Test_Precision@20`) and additionally extracts `BEST_Val_Recall_Peak_Epoch` / `BEST_Val_NDCG_Peak_Epoch` for the post-hoc audit.
Runs over-train after convergence	`early_stopping_patience` default lowered from **30** to **20**; flip `BACK_TO_PATIENCE_30 = True` to reproduce the previous behaviour.

Table 1: Five defects in the previous M1.5 sweep cell and their corresponding fixes in the patched cell.

Reporting protocol

When you summarise an M1.5 outcome, quote exclusively `BEST_Test_Recall@20` (the val-peak snapshot). Do *not* cite Recall@20 printed on epoch 84 or 219 in the middle of a run – those numbers are explicitly *not* the model that would be deployed under the `restore_best=1` early-stopping policy.

4 Part 2 – Append a Post-hoc Analysis Cell

Append the script in Appendix B (`cell31_m15_analysis.py`) as the **new last cell** of the notebook (preceded by a level-2 Markdown header titled "Wave 1 / M1.5 Sweep – Post-hoc Analysis"). The cell is read-only: it never launches `train.py`; it only inspects log artefacts already produced by the sweep cell.

What it does

1. Globs every directory under `../<dataset>/damps_*` whose name matches the regex below, and locates its `.txt` logger output:

```
damps_..._seed=<seed>_m15_logq_<mode>_s<scale>_seed<seed>
```

2. Parses each log for the trailing summary lines emitted by `train.py` at the end of training, namely:

```
BEST_Test_Recall@20 / NDCG@20 / Precision@20
BEST_Val_Recall@20 / NDCG@20
BEST_Val_Recall_Peak_Epoch
BEST_Val_NDCG_Peak_Epoch
```

plus every per-epoch diagnostic line of the form:

```
Epoch N [..s + ..s]: loss=.. recall@10=.. recall@20=.. ndcg@20=..
```

3. Builds a per-run audit table containing, for every (mode, scale, seed), the `BEST_Test_Recall@20`, the validation-recall peak epoch, the last epoch reached, and the margin `(last - peak)`. Any run whose margin is below 15 epochs is flagged for inspection (this would be evidence that early stopping cut the run short of its real peak).
4. Re-computes the M1.5 gate purely from the parsed logs, as a sanity cross-check of the in-memory `all_runs` dictionary built by the sweep cell.
5. Plots, for every `logq_scale`, the val/recall@20 curve of each seed on a shared axis; overlays a dotted vertical line at the recall peak, plus horizontal lines at the rollback gate (0.0890) and the M1.5 gate (0.0925). The figure is saved as `../<dataset>/m15_logq_val_recall_curves.png`.
6. Persists the parsed run dictionaries and aggregate to `../<dataset>/m15_logq_parsed_logs.json`.

Interpreting the visual output

For every (scale, seed) curve, look for two patterns:

- ✓ **Healthy:** the curve rises monotonically (with validation noise), reaches its peak around the middle or back third of the run, then plateaus or drops; the last epoch is at least 15–20 epochs past the peak. The dotted vertical line sits comfortably to the left of the right edge. `restore_best=1` guarantees that the *deployed* model is the one at the peak, not the final epoch.
- ! **Suspect:** the peak is within the last 5–10 epochs of the run, i.e. the curve still has positive slope when training ended. This would indicate the run was halted prematurely. The cell flags such cases in the console output.

5 Part 3 – Recipe to Execute the Patch

1. Open the notebook `Local_Random_seeds_train_mmhcl_clothing_colab_original.ipynb` in JupyterLab.
2. Replace the contents of the final code cell with `cell30_m15_fixed.py` (Appendix A).
3. Append a new Markdown header (`## 8. Wave 1 / M1.5 Sweep -- Post-hoc Analysis`) and a new code cell containing `cell31_m15_analysis.py` (Appendix B).
4. Save the notebook as `Local_Random_seeds_train_mmhcl_clothing_colab_rev54_wave1_fixed.ipynb`.
5. Re-run the sweep cell (the original took ∼20 runs × 4–6h on RTX 5090). Once it finishes, run the analysis cell on the same kernel.
6. Quote `BEST_Test_Recall@20` (not mid-run printouts) when reporting the M1.5 gate verdict, and attach `m15_logq_val_recall_curves.png` as evidence that no run was terminated prematurely.

A Appendix A – Patched Cell (`cell30_m15_fixed.py`)

```
# ======================================================================
# Section 8.2 -- Wave 1 / M1.5 sweep driver (FIXED for rev54)
# Matrix: logq_scale x logq_mode x seed →calls train.py per config
# Gate: mean BEST_Test_Recall@20 ≥0.0925 AND every seed ≥0.0890
#
```

```python
# FIXES vs the previous notebook revision
# (1) Real LogQ activation: --enable_logq 1 + --logq_mode/--logq_beta/
# --logq_scale/--logq_clip are now ACTUALLY appended to every cmd.
# (2) Seeds are coerced via int(round(float(s))) before being passed as
# --seed, eliminating the "seed=1.40632241e+09" float-truncation
# reproducibility bug.
# (3) BEST_Test_* (snapshot at the val-recall peak epoch) is the only
# metric reported for the M1.5 gate, NOT mid-run epoch printouts.
# (4) ablation_target now encodes (mode, scale, seed) so per-run log
# directories do not collide across the sweep matrix.
# (5) early_stopping_patience lowered from 30 →20 to terminate cleanly
# after convergence (optional; flip BACK_TO_PATIENCE_30 = True to
# reproduce the previous behaviour exactly).
# ======================================================================
from __future__ import annotations

import os
import re
import sys
import json
import time
import random
import subprocess
from typing import Any

import wandb

# ---- 0. Resolve project dirs (reuse earlier-cell globals if present) ----
try:
    _DAMPS_DIR = DAMPS_DIR
except NameError:
    _DAMPS_DIR = os.getcwd()
try:
    _PYTHON = PYTHON_EXE
except NameError:
    _PYTHON = sys.executable
try:
    _DATASET = dataset
except NameError:
    _DATASET = "Clothing"
try:
    _WB_PROJECT = wandb_project
except NameError:
    _WB_PROJECT = "damps-mmhcl-clothing"
try:
    _WB_ENTITY = wandb_entity
except NameError:
    _WB_ENTITY = ""

if os.path.normpath(os.getcwd()) != os.path.normpath(_DAMPS_DIR):
    os.chdir(_DAMPS_DIR)

# ---- 1. Sweep matrix definition -------------------------------------
LOGQ_SCALES: list[float] = [0.05, 0.1, 0.3, 1.0] # spec-mandated scale sweep
LOGQ_MODES: list[str] = ["laplace"] # add "raw","sqrt" →24 runs
N_SEEDS: int = 5 # 4 scales x 1 mode x 5 = 20 runs
LOGQ_BETA: float = 1.0
LOGQ_CLIP: float = 5.0

# M1.5 acceptance gate (rev54 Section 8.2 / Section 6)
GATE_MEAN_RECALL: float = 0.0925
GATE_MIN_RECALL: float = 0.0890

# Optional: revert patience to 30 to reproduce the previous notebook exactly
BACK_TO_PATIENCE_30: bool = False
PATIENCE: int = 30 if BACK_TO_PATIENCE_30 else 20
MIN_EPOCHS: int = 75

# ---- 1a. Seed coercion: ALWAYS int (fixes the 1.40e+09 float bug) ----
def _as_int_seed(s: Any) →int:
    """Tolerate floats / strings / numpy scalars; emit a clean Python int."""
    return int(round(float(s)))

try:
```

```
    SEEDS = [_as_int_seed(s) for s in list(seeds)[:N_SEEDS]] # type: ignore[name-defined] # noqa: F821
    if len(SEEDS) < N_SEEDS:
        raise NameError
except NameError:
    random.seed(2026)
    SEEDS = [random.randint(1, 2_147_483_646) for _ in range(N_SEEDS)]
print(f"[M1.5] seeds (int) = {SEEDS}")
print(f"[M1.5] patience = {PATIENCE} (BACK_TO_PATIENCE_30={BACK_TO_PATIENCE_30})")
print(f"[M1.5] W&B project = {_WB_PROJECT!r} entity={_WB_ENTITY!r}")

# ---- 2. rev45 apc_off_combined backbone flags (only delta = LogQ) -----
# Identical to the Round-2 apc_off_combined cell, except early_stopping
# patience is lowered to 20 by default.
BACKBONE_FLAGS: list[str] = [
    "--dataset", str(_DATASET),
    "--gpu_id", "0",
    "--epoch", "250",
    "--verbose", "5",
    "--batch_size", "1024",
    "--lr", "0.0001",
    "--regs", "0.001",
    "--clip_grad_norm", "1.0",
    "--embed_size", "64",
    "--topk", "5",
    "--core", "5",
    "--User_layers", "3",
    "--Item_layers", "2",
    "--user_loss_ratio", "0.03",
    "--item_loss_ratio", "0.07",
    "--temperature", "0.3",
    "--learnable_tau", "0",
    "--early_stopping_patience", str(PATIENCE),
    "--early_stopping_min_epochs", str(MIN_EPOCHS),
    "--early_stopping_min_delta", "0.0001",
    "--early_stopping_mode", "max",
    "--early_stopping_restore_best", "1",
    "--use_reduce_lr", "1",
    "--reduce_lr_factor", "0.5",
    "--reduce_lr_patience", "3",
    "--reduce_lr_min", "1e-06",
    # apc_off_combined backbone
    "--damps_apc", "0",
    "--damps_avrf", "0",
    "--damps_imcf", "1",
    "--damps_permutation_fft", "0",
    "--damps_soft_routing", "1",
    "--damps_momentum", "1",
    "--damps_data_driven_prior", "1",
    "--damps_num_categories", "10",
    "--damps_warmup_epochs", "10",
    "--rebuild_R", "5",
    "--faiss_threshold", "60000",
    "--faiss_use_gpu", "1",
    "--knn_chunk_size", "4096",
    "--knn_efsearch", "64",
    "--use_amp", "1",
]

# ---- 3. Metric parser: prefer BEST_Test_* (val-peak snapshot) --------
# train.py emits:
# BEST_Val_Recall@10 / @20
# BEST_Val_Recall_Peak_Epoch
# BEST_Val_NDCG@10 / @20
# BEST_Val_NDCG_Peak_Epoch
# BEST_Test_Recall@20 / Precision@20 / NDCG@20 <-- gate uses these
_S = r"[^0-9\-]*(-?[0-9]*\.?[0-9]+)"
_PAT = {
    "recall@20": re.compile(r"BEST_Test_Recall@20" + _S),
    "ndcg@20": re.compile(r"BEST_Test_NDCG@20" + _S),
    "precision@20": re.compile(r"BEST_Test_Precision@20" + _S),
    "val_recall@20": re.compile(r"BEST_Val_Recall@20" + _S),
    "val_ndcg@20": re.compile(r"BEST_Val_NDCG@20" + _S),
    "val_recall_peak_epoch":re.compile(r"BEST_Val_Recall_Peak_Epoch" + _S),
    "val_ndcg_peak_epoch": re.compile(r"BEST_Val_NDCG_Peak_Epoch" + _S),
```

```
}

def _parse_metrics(text: str) →dict[str, float]:
    out: dict[str, float] = {}
    for k, pat in _PAT.items():
        hits = pat.findall(text)
        if hits:
            out[k] = float(hits[-1]) # last write = final best snapshot
    return out

# ---- 4. Run one (mode, scale, seed) configuration --------------------
def run_one(mode: str, scale: float, seed: int) →dict[str, Any]:
    """Launch one train.py subprocess; returns a result dict."""
    seed_int = int(seed)
    # Encode mode/scale/seed into the ablation_target so per-run log
    # directories (../<dataset>/damps_..._seed=<seed>_<ablation_target>/)
    # do not collide across sweep configurations.
    abl_tag = f"m15_logq_{mode}_s{scale}_seed{seed_int}"
    run_name = abl_tag
    cmd: list[str] = [
        _PYTHON, "train.py",
        *BACKBONE_FLAGS,
        "--seed", str(seed_int), # <-- ALWAYS int, never 1.40e+09
        # ---- Wave 1 LogQ activation (the only delta vs rev45) ----
        "--enable_logq", "1",
        "--logq_mode", str(mode),
        "--logq_beta", str(LOGQ_BETA),
        "--logq_scale", str(scale),
        "--logq_clip", str(LOGQ_CLIP),
        # ---- W&B per-run logging ----
        "--use_wandb", "1",
        "--wandb_project", _WB_PROJECT,
        "--wandb_entity", _WB_ENTITY,
        "--wandb_run_name", run_name,
        "--ablation_target", abl_tag,
    ]
    t0 = time.time()
    env = os.environ.copy()
    env["PYTHONIOENCODING"] = "utf-8"
    env["PYTHONUNBUFFERED"] = "1"
    proc = subprocess.run(
        cmd, capture_output=True, text=True, cwd=_DAMPS_DIR, env=env
    )
    dt = time.time() - t0
    log = (proc.stdout or "") + "\n" + (proc.stderr or "")
    m = _parse_metrics(log)
    ok = ("recall@20" in m) and (proc.returncode == 0)
    if not ok:
        print(f" [WARN] mode={mode} scale={scale} seed={seed_int} "
              f"rc={proc.returncode}\n{log[-400:]}")
    return {
        "mode": mode,
        "scale": scale,
        "seed": seed_int,
        "run_name": run_name,
        "ablation_target": abl_tag,
        "recall@20": m.get("recall@20"),
        "ndcg@20": m.get("ndcg@20"),
        "precision@20": m.get("precision@20"),
        "val_recall@20": m.get("val_recall@20"),
        "val_ndcg@20": m.get("val_ndcg@20"),
        "val_recall_peak_epoch": int(m["val_recall_peak_epoch"])
                                 if "val_recall_peak_epoch" in m else None,
        "val_ndcg_peak_epoch": int(m["val_ndcg_peak_epoch"])
                               if "val_ndcg_peak_epoch" in m else None,
        "runtime_s": round(dt, 1),
        "ok": ok,
    }

# ---- 5. Execute the full matrix --------------------------------------
all_runs: list[dict[str, Any]] = []
total = len(LOGQ_MODES) * len(LOGQ_SCALES) * len(SEEDS)
idx = 0
print("=" * 72)
```

```
print(f"M1.5 SWEEP: {total} runs "
      f"({len(LOGQ_MODES)} mode x {len(LOGQ_SCALES)} scale x {len(SEEDS)} seed)")
print("=" * 72)
for mode in LOGQ_MODES:
    for scale in LOGQ_SCALES:
        for seed in SEEDS:
            idx += 1
            print(f"[{idx}/{total}] mode={mode} scale={scale} "
                  f"seed={int(seed)} ...", flush=True)
            all_runs.append(run_one(mode, scale, seed))

# ---- 6. Aggregate per (mode, scale) and evaluate the gate ------------
def _mean(xs): return sum(xs) / len(xs) if xs else float("nan")
def _std(xs):
    if len(xs) < 2:
        return 0.0
    m = _mean(xs)
    return (sum((x - m) ** 2 for x in xs) / (len(xs) - 1)) ** 0.5

summary: list[dict[str, Any]] = []
for mode in LOGQ_MODES:
    for scale in LOGQ_SCALES:
        rs = [r["recall@20"] for r in all_runs
              if r["mode"] == mode and r["scale"] == scale
              and r["recall@20"] is not None]
        ns = [r["ndcg@20"] for r in all_runs
              if r["mode"] == mode and r["scale"] == scale
              and r["ndcg@20"] is not None]
        if not rs:
            continue
        mean_r, min_r = _mean(rs), min(rs)
        passed = (mean_r ≥GATE_MEAN_RECALL) and (min_r ≥GATE_MIN_RECALL)
        summary.append({
            "mode": mode,
            "scale": scale,
            "n": len(rs),
            "recall_mean": round(mean_r, 4),
            "recall_std": round(_std(rs), 4),
            "recall_min": round(min_r, 4),
            "ndcg_mean": round(_mean(ns), 4),
            "ndcg_std": round(_std(ns), 4),
            "M1.5_pass": passed,
        })

# ---- 7. Console report -----------------------------------------------
print("\n" + "=" * 72)
print("M1.5 GATE REPORT (mean BEST_Test_R@20 ≥0.0925 AND min ≥0.0890)")
print("=" * 72)
hdr = (f"{'mode':<9}{'scale':>7}{'n':>4}{'R@20 mean':>12}{'+/-':>9}"
       f"{'R@20 min':>11}{'NDCG mean':>11}{'PASS':>7}")
print(hdr)
print("-" * len(hdr))
for s in summary:
    print(f"{s['mode']:<9}{s['scale']:>7}{s['n']:>4}"
          f"{s['recall_mean']:>12.4f}{s['recall_std']:>9.4f}"
          f"{s['recall_min']:>11.4f}{s['ndcg_mean']:>11.4f}"
          f"{('YES' if s['M1.5_pass'] else 'no'):>7}")

any_pass = any(s["M1.5_pass"] for s in summary)
verdict = ("PASS" if any_pass else "FAIL")
print("\n[M1.5 VERDICT]",
      "PASS -- at least one (mode,scale) cleared the gate; "
      "promote that config to Wave 2 (M1 / SimGCL)."
      if any_pass else
      "FAIL -- no (mode,scale) cleared the gate; "
      "inspect logq_scale/clip before proceeding.")

# ---- 8. Persist raw + summary JSON for audit trail -------------------
out_dir = os.path.abspath(os.path.join("..", _DATASET))
os.makedirs(out_dir, exist_ok=True)
runs_path = os.path.join(out_dir, "m15_logq_sweep_runs.json")
summary_path = os.path.join(out_dir, "m15_logq_sweep_summary.json")
with open(runs_path, "w", encoding="utf-8") as f:
    json.dump(all_runs, f, indent=2)
```

```
with open(summary_path, "w", encoding="utf-8") as f:
    json.dump(summary, f, indent=2)
print(f"\nSaved →{runs_path}")
print(f"Saved →{summary_path}")

# ---- 9. W&B sweep-level summary run ----------------------------------
print("\n[W&B] Logging sweep-level summary run ...")
wb_run = wandb.init(
    project=_WB_PROJECT,
    entity=_WB_ENTITY if _WB_ENTITY else None,
    name="m15_logq_sweep_summary",
    group="wave1_m15_logq",
    tags=["wave1", "m15", "logq", "sweep_summary"],
    job_type="sweep_summary",
    config={
        "phase": "wave1_m15",
        "logq_modes": LOGQ_MODES,
        "logq_scales": LOGQ_SCALES,
        "logq_beta": LOGQ_BETA,
        "logq_clip": LOGQ_CLIP,
        "seeds": SEEDS,
        "n_seeds": N_SEEDS,
        "total_runs": total,
        "gate_mean_recall": GATE_MEAN_RECALL,
        "gate_min_recall": GATE_MIN_RECALL,
        "dataset": _DATASET,
        "backbone": "apc_off_combined",
        "temperature": 0.3,
        "patience": PATIENCE,
    },
    reinit=True,
)
cols = ["mode", "scale", "seed", "run_name",
        "recall@20", "ndcg@20", "precision@20",
        "val_recall@20", "val_ndcg@20",
        "val_recall_peak_epoch", "val_ndcg_peak_epoch",
        "runtime_s", "ok"]
runs_table = wandb.Table(columns=cols)
for r in all_runs:
    runs_table.add_data(*[r.get(c) for c in cols])
wb_run.log({"m15/all_runs": runs_table})

agg_cols = ["mode", "scale", "n",
            "recall_mean", "recall_std", "recall_min",
            "ndcg_mean", "ndcg_std", "M1.5_pass"]
agg_table = wandb.Table(columns=agg_cols)
for s in summary:
    agg_table.add_data(*[s.get(c) for c in agg_cols])
    tag = f"{s['mode']}_s{s['scale']}"
    wb_run.log({
        f"m15/{tag}/recall_mean": s["recall_mean"],
        f"m15/{tag}/recall_std": s["recall_std"],
        f"m15/{tag}/recall_min": s["recall_min"],
        f"m15/{tag}/ndcg_mean": s["ndcg_mean"],
        f"m15/{tag}/gate_pass": int(s["M1.5_pass"]),
    })
wb_run.log({"m15/aggregate_table": agg_table})

best = max(
    summary,
    key=lambda s: (s["recall_mean"], -s["recall_std"]),
    default=None,
)
wb_run.summary.update({
    "m15/gate_passed": any_pass,
    "m15/verdict": verdict,
    "m15/best_recall_mean": (best["recall_mean"] if best else None),
    "m15/best_scale": (best["scale"] if best else None),
    "m15/best_mode": (best["mode"] if best else None),
    "m15/n_configs_passed": sum(1 for s in summary if s["M1.5_pass"]),
    "m15/n_configs_total": len(summary),
})

artifact = wandb.Artifact(
```

```
    name=f"m15_logq_sweep_{_DATASET}",
    type="sweep_results",
    description="Raw per-run results + per-(mode,scale) aggregates for M1.5 LogQ sweep.",
    metadata={"dataset": _DATASET, "verdict": verdict},
)
artifact.add_file(runs_path, name="m15_logq_sweep_runs.json")
artifact.add_file(summary_path, name="m15_logq_sweep_summary.json")
wb_run.log_artifact(artifact)
wb_run.finish()
print(f"[W&B] Sweep summary run logged: {wb_run.url}")
```

B Appendix B – Post-hoc Analysis Cell (`cell31_m15_analysis.py`)

```python
# ============================================================================
# Section 8.2 -- Wave 1 / M1.5 sweep ANALYSIS (post-hoc, read-only)
# For each (mode, scale, seed) directory under ../<dataset>/, extract:
# * BEST_Test_Recall@20 / NDCG@20 / Precision@20 (val-peak snapshot)
# * BEST_Val_Recall@20 + Peak_Epoch (early-stop proof)
# * Full per-epoch val/recall@20 curve (early-stop visual)
# Plot val-recall curves so you can visually confirm NO run was cut
# short of its recall peak.
#
# Required upstream artifacts (produced by the fixed sweep cell):
# ../<dataset>/m15_logq_sweep_runs.json (optional cross-check)
# ../<dataset>/damps_..._seed=<seed>_m15_logq_<mode>_s<scale>_seed<seed>/
# *.txt (per-epoch logger output)
# ============================================================================
from __future__ import annotations

import os
import re
import json
import glob
import math
from typing import Any, Optional

import numpy as np
import matplotlib.pyplot as plt

# ---- 0. Resolve dirs ------------------------------------------------
try:
    _DAMPS_DIR = DAMPS_DIR
except NameError:
    _DAMPS_DIR = os.getcwd()
try:
    _DATASET = dataset
except NameError:
    _DATASET = "Clothing"

if os.path.normpath(os.getcwd()) != os.path.normpath(_DAMPS_DIR):
    os.chdir(_DAMPS_DIR)

LOG_ROOT = os.path.abspath(os.path.join("..", _DATASET))
print(f"[M1.5/analysis] scanning {LOG_ROOT}")

# ---- 1. Discover run directories from ablation_target naming ---------
# Pattern produced by the fixed sweep cell:
# damps_..._seed=<seed>_m15_logq_<mode>_s<scale>_seed<seed>/<same>.txt
DIR_RE = re.compile(
    r"damps_.*?_seed=(?P<seed>\d+)_m15_logq_(?P<mode>[a-zA-Z0-9]+)_s(?P<scale>[0-9.]+)_seed\d+$"
)

run_dirs: list[dict[str, Any]] = []
for d in sorted(glob.glob(os.path.join(LOG_ROOT, "damps_*"))):
    if not os.path.isdir(d):
        continue
    name = os.path.basename(d.rstrip("/"))
    m = DIR_RE.match(name)
    if not m:
        continue
    txt_files = glob.glob(os.path.join(d, "*.txt"))
    if not txt_files:
        continue
    run_dirs.append({
        "dir": d,
        "txt": txt_files[0],
        "mode": m.group("mode"),
        "scale": float(m.group("scale")),
        "seed": int(m.group("seed")),
    })
print(f"[M1.5/analysis] found {len(run_dirs)} M1.5 LogQ run directories")
if not run_dirs:
    print(" (no logs yet -- run the fixed sweep cell first)")

```

```
# ---- 2. Regex parsers ------------------------------------------------
_S = r"[^0-9\-]*(-?[0-9]*\.?[0-9]+)"
PAT_BEST = {
    "test_recall@20": re.compile(r"BEST_Test_Recall@20" + _S),
    "test_ndcg@20": re.compile(r"BEST_Test_NDCG@20" + _S),
    "test_precision@20": re.compile(r"BEST_Test_Precision@20" + _S),
    "val_recall@20": re.compile(r"BEST_Val_Recall@20" + _S),
    "val_ndcg@20": re.compile(r"BEST_Val_NDCG@20" + _S),
    "val_recall_peak_epoch": re.compile(r"BEST_Val_Recall_Peak_Epoch" + _S),
    "val_ndcg_peak_epoch": re.compile(r"BEST_Val_NDCG_Peak_Epoch" + _S),
}
# Per-epoch line from train.py Trainer.train():
# Epoch 17 [12.3s + 4.5s]: loss=0.12345 recall@10=0.04210 recall@20=0.07815 ndcg@20=0.03812
PAT_EPOCH = re.compile(
    r"Epoch\s+(\d+)\s*\[[^\]]+\]:\s*loss=([\d.]+)\s+"
    r"recall@\d+=([\d.]+)\s+recall@(\d+)=([\d.]+)\s+ndcg@\d+=([\d.]+)"
)
# Early-stop marker emitted by train.py
PAT_ES_TRIGGER = re.compile(r"#####\s*Early stop triggered\s*#####")


def parse_log(path: str) →dict[str, Any]:
    """Return BEST_* snapshot, per-epoch curves, and early-stop flag."""
    with open(path, "r", encoding="utf-8", errors="replace") as f:
        text = f.read()
    best: dict[str, float] = {}
    for k, pat in PAT_BEST.items():
        hits = pat.findall(text)
        if hits:
            best[k] = float(hits[-1])
    epochs: list[int] = []
    val_rec20: list[float] = []
    val_ndcg20: list[float] = []
    losses: list[float] = []
    for m in PAT_EPOCH.finditer(text):
        epochs.append(int(m.group(1)))
        losses.append(float(m.group(2)))
        val_rec20.append(float(m.group(5)))
        val_ndcg20.append(float(m.group(6)))
    return {
        "best": best,
        "epochs": epochs,
        "loss": losses,
        "val_rec20": val_rec20,
        "val_ndcg20": val_ndcg20,
        "early_stop_triggered": bool(PAT_ES_TRIGGER.search(text)),
        "last_epoch": (epochs[-1] if epochs else None),
    }


# ---- 3. Parse all logs -----------------------------------------------
rows: list[dict[str, Any]] = []
for r in run_dirs:
    p = parse_log(r["txt"])
    best = p["best"]
    peak = int(best.get("val_recall_peak_epoch", -1))
    last = p["last_epoch"]
    margin = (last - peak) if (last is not None and peak ≥0) else None
    rows.append({
        "mode": r["mode"],
        "scale": r["scale"],
        "seed": r["seed"],
        "BEST_Test_R@20": best.get("test_recall@20"),
        "BEST_Test_NDCG@20":best.get("test_ndcg@20"),
        "BEST_Val_R@20": best.get("val_recall@20"),
        "val_peak_epoch": peak if peak ≥0 else None,
        "last_epoch": last,
        "epochs_past_peak": margin,
        "early_stop": p["early_stop_triggered"],
        "n_val_points": len(p["val_rec20"]),
        "_curve": p,
    })

# ---- 4. Tabular report (BEST_Test + early-stop margin) ---------------
```

```
print("\n" + "=" * 92)
print("M1.5 per-run audit (sorted by mode, scale, seed)")
print("=" * 92)
hdr = (f"{'mode':<9}{'scale':>7}{'seed':>12}{'BEST_Test_R@20':>17}"
       f"{'BEST_Val_R@20':>16}{'peak_ep':>9}{'last_ep':>9}"
       f"{'margin':>8}{'ES?':>5}")
print(hdr); print("-" * len(hdr))
for x in sorted(rows, key=lambda r: (r["mode"], r["scale"], r["seed"])):
    print(f"{x['mode']:<9}{x['scale']:>7}{x['seed']:>12}"
          f"{(x['BEST_Test_R@20'] or float('nan')):>17.4f}"
          f"{(x['BEST_Val_R@20'] or float('nan')):>16.4f}"
          f"{(x['val_peak_epoch'] if x['val_peak_epoch'] is not None else -1):>9}"
          f"{(x['last_epoch'] if x['last_epoch'] is not None else -1):>9}"
          f"{(x['epochs_past_peak'] if x['epochs_past_peak'] is not None else -1):>8}"
          f"{('Y' if x['early_stop'] else 'n'):>5}")

# Flag suspicious cases: peak too close to last epoch (< PATIENCE)
SUSPECT_MARGIN: int = 15
suspects = [
    x for x in rows
    if x["epochs_past_peak"] is not None and x["epochs_past_peak"] < SUSPECT_MARGIN
]
if suspects:
    print(f"\n[WARN] {len(suspects)} run(s) ended within {SUSPECT_MARGIN} epochs of "
          f"their val-recall peak -- inspect curves to rule out premature stop:")
    for x in suspects:
        print(f"  mode={x['mode']} scale={x['scale']} seed={x['seed']} "
              f"peak={x['val_peak_epoch']} last={x['last_epoch']} "
              f"margin={x['epochs_past_peak']}")
else:
    print(f"\n[OK] all runs ran ≥{SUSPECT_MARGIN} epochs past their val-recall peak.")

# ---- 5. Per-(mode, scale) aggregate using BEST_Test only --------------
def _agg(vals: list[Optional[float]]) →tuple[float, float, float]:
    v = [x for x in vals if x is not None]
    if not v:
        return float("nan"), float("nan"), float("nan")
    m = sum(v) / len(v)
    s = math.sqrt(sum((x - m) ** 2 for x in v) / (len(v) - 1)) if len(v) ≥2 else 0.0
    return m, s, min(v)

print("\n" + "=" * 92)
print("M1.5 aggregate from PARSED LOGS (BEST_Test_Recall@20 only)")
print("=" * 92)
key = lambda r: (r["mode"], r["scale"])
configs = sorted(set(key(r) for r in rows))
print(f"{'mode':<9}{'scale':>7}{'n':>4}{'R@20 mean':>12}{'+/-':>9}"
      f"{'R@20 min':>11}{'NDCG mean':>11}{'PASS':>7}")
print("-" * 64)
GATE_MEAN: float = 0.0925
GATE_MIN: float = 0.0890
log_summary: list[dict[str, Any]] = []
for mode, scale in configs:
    sub = [r for r in rows if r["mode"] == mode and r["scale"] == scale]
    rm, rs, mn = _agg([r["BEST_Test_R@20"] for r in sub])
    nm, _, _ = _agg([r["BEST_Test_NDCG@20"] for r in sub])
    passed = (rm ≥GATE_MEAN) and (mn ≥GATE_MIN)
    log_summary.append({
        "mode": mode, "scale": scale, "n": len(sub),
        "recall_mean": round(rm, 4), "recall_std": round(rs, 4),
        "recall_min": round(mn, 4), "ndcg_mean": round(nm, 4),
        "M1.5_pass": bool(passed),
    })
    print(f"{mode:<9}{scale:>7}{len(sub):>4}{rm:>12.4f}{rs:>9.4f}"
          f"{mn:>11.4f}{nm:>11.4f}{('YES' if passed else 'no'):>7}")

# ---- 6. Visual confirmation: val-recall@20 curves ---------------------
if rows:
    by_scale: dict[float, list[dict[str, Any]]] = {}
    for r in rows:
        by_scale.setdefault(r["scale"], []).append(r)
    n_scales = len(by_scale)
    ncols = min(2, n_scales)
    nrows = math.ceil(n_scales / ncols)
```

```
    fig, axes = plt.subplots(nrows, ncols,
                             figsize=(7.0 * ncols, 4.2 * nrows),
                             squeeze=False)
    for ax_idx, (scale, group) in enumerate(sorted(by_scale.items())):
        ax = axes[ax_idx // ncols][ax_idx % ncols]
        for x in sorted(group, key=lambda r: r["seed"]):
            curve = x["_curve"]
            if not curve["epochs"]:
                continue
            ax.plot(curve["epochs"], curve["val_rec20"],
                    linewidth=1.4, alpha=0.85,
                    label=f"seed={x['seed']} (peak ep={x['val_peak_epoch']}, "
                          f"BEST_Test={(x['BEST_Test_R@20'] or 0):.4f})")
            if x["val_peak_epoch"] is not None:
                ax.axvline(x["val_peak_epoch"], color="grey",
                           linestyle=":", linewidth=0.7, alpha=0.6)
        ax.axhline(GATE_MIN, color="red", linestyle="--",
                   linewidth=0.9, alpha=0.7, label=f"rollback {GATE_MIN}")
        ax.axhline(GATE_MEAN, color="green", linestyle="--",
                   linewidth=0.9, alpha=0.7, label=f"M1.5 gate {GATE_MEAN}")
        ax.set_title(f"M1.5 LogQ -- scale={scale}")
        ax.set_xlabel("epoch")
        ax.set_ylabel("val/recall@20")
        ax.grid(True, alpha=0.3)
        ax.legend(fontsize=7, loc="lower right")
    # blank out any spare subplot
    for j in range(len(by_scale), nrows * ncols):
        axes[j // ncols][j % ncols].axis("off")
    fig.suptitle("M1.5 LogQ sweep -- val/recall@20 vs epoch "
                 "(dotted = val-recall peak; restore_best=1 means best is recovered)",
                 y=1.02, fontsize=11, fontweight="bold")
    plt.tight_layout()
    fig_path = os.path.join(LOG_ROOT, "m15_logq_val_recall_curves.png")
    plt.savefig(fig_path, dpi=140, bbox_inches="tight")
    plt.show()
    print(f"[plot] saved →{fig_path}")

# ---- 7. Persist parsed analysis to JSON ------------------------------
parsed_path = os.path.join(LOG_ROOT, "m15_logq_parsed_logs.json")
to_dump = []
for r in rows:
    rr = {k: v for k, v in r.items() if k != "_curve"}
    rr["epochs"] = r["_curve"]["epochs"]
    rr["val_rec20"] = r["_curve"]["val_rec20"]
    rr["val_ndcg20"] = r["_curve"]["val_ndcg20"]
    to_dump.append(rr)
with open(parsed_path, "w", encoding="utf-8") as f:
    json.dump({"runs": to_dump, "summary": log_summary}, f, indent=2)
print(f"[json] saved →{parsed_path}")
```